KYUUBI × APACHE ARROW

Arrow Flight SQL as a Kyuubi frontend

Story of this talk: Kyuubi already multiplexes engines behind frontends → analytics clients want columnar results → Flight SQL is the wire for that → then the implementation details (lifecycle, streaming, auth, HA).

Act	What we cover
1. Kyuubi today	Gateway, frontends, dispatch
2. Why Arrow / Flight SQL	Columnar gap → Flight transport → SQL commands
3. What we built	Lifecycle, config, streaming, types, auth, HA, ops

- Stack: **Java 17**, **Arrow 16**, gRPC Flight SQL · default port `10299`

01 · WHAT KYUUBI IS

Kyuubi is a multi-tenant SQL gateway

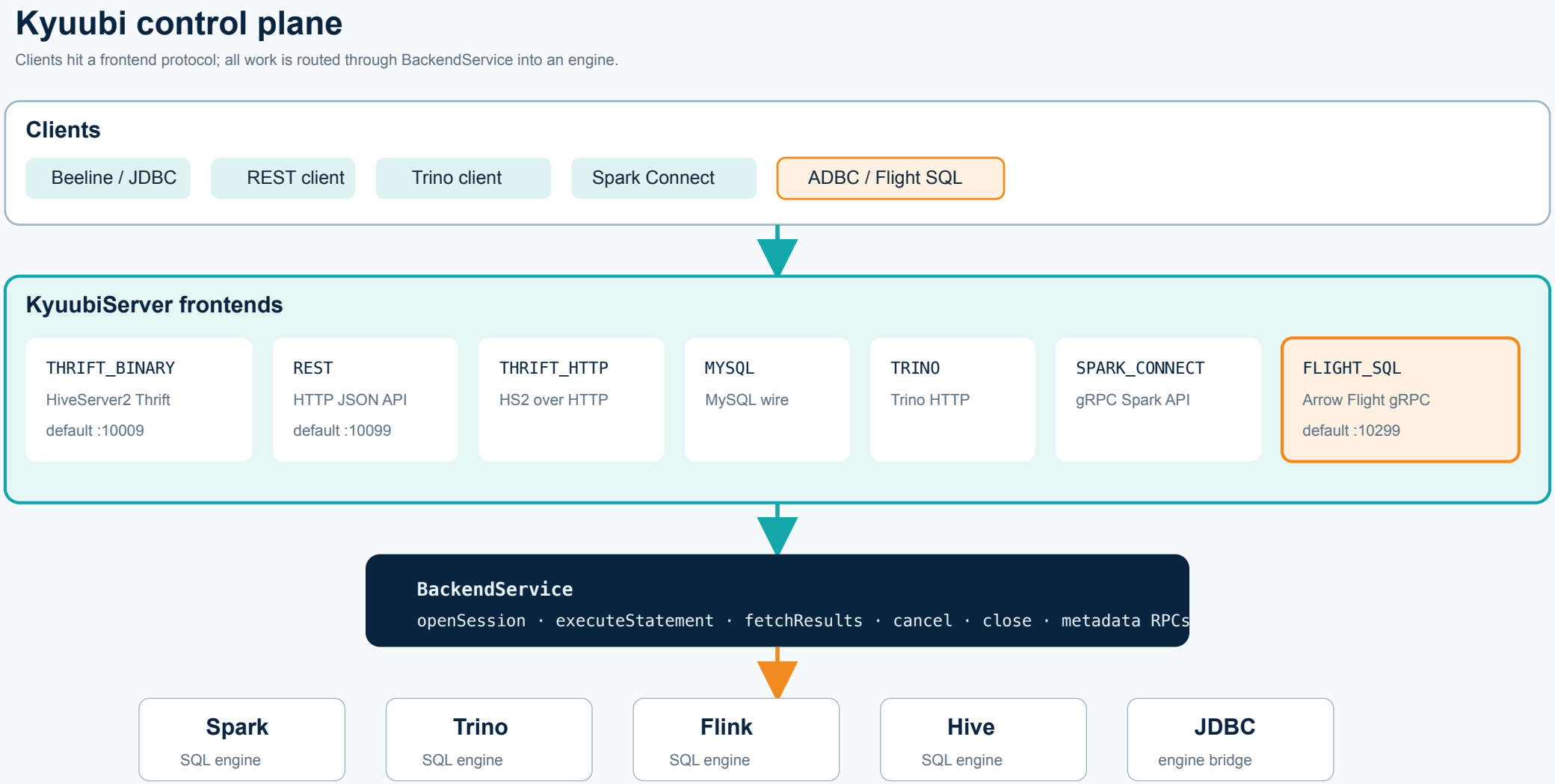
Kyuubi sits between SQL clients and query engines. It owns the **control plane**:

1. **Sessions** – authenticate user, create/reuse engine session
2. **Operations** – execute SQL, track status, cancel, close
3. **Engine lifecycle** – launch Spark / Trino / Flink / Hive / JDBC engines
4. **Cross-cutting concerns** – HA discovery, metrics, result formatting

Clients do **not** talk to engines directly. They talk to a **frontend protocol**. Every frontend calls the shared `BackendService`.

01 · VISUAL

Control plane: clients → frontends → BackendService → engines



02 · EXISTING FRONTENDS

Frontends already implemented

STILL ACT 1 — SAME BACKENDSERVICE, MANY WIRES

Configured by `kyuubi.frontend.protocols` (`FrontendProtocols` enum):

Protocol	Role
`THRIFT_BINARY`	Default HiveServer2 JDBC / Beeline path
`REST`	Default Kyuubi REST API
`THRIFT_HTTP`	HS2 over HTTP
`MYSQL`	MySQL text protocol (experimental)
`TRINO`	Trino HTTP (experimental)
`SPARK_CONNECT`	Spark Connect gRPC (experimental)
`FLIGHT_SQL`	**New** Arrow Flight SQL gRPC frontend

Defaults are `THRIFT_BINARY,REST`. Everything else is opt-in by adding the enum name to the protocol list.

02 · VISUAL

Protocol matrix: transport, client, result shape

Frontend protocol matrix

What already ships in Kyuubi, and where Flight SQL fits.

Protocol	Transport	Typical client	Result shape	Default
THRIFT_BINARY	TCP Thrift	Beeline / JDBC	rows / Arrow TRowSet	yes
REST	HTTP JSON	REST / admin clients	JSON DTOs	yes
THRIFT_HTTP	HTTP Thrift	HS2 HTTP clients	rows	opt-in
MYSQL / TRINO	MySQL / HTTP	mysql / Trino clients	rows	opt-in
SPARK_CONNECT	gRPC	Spark Connect client	Spark API results	opt-in
FLIGHT_SQL	gRPC Flight	ADBC / FlightSqlClient	Arrow IPC batches	opt-in · NEW

Enable with: kyuubi.frontend.protocols=...,FLIGHT_SQL

Flight SQL reuses the same BackendService path as Thrift/REST. The difference is the wire protocol and result encoding.

03 · HOW FRONTENDS WORK

Dispatch is config → enum → service class

STILL ACT 1 – HOW A PROTOCOL BECOMES A RUNNING SERVICE

There is **no** separate `kyuubi.flight.sql.enabled` boolean as the source of truth.

```
kyuubi.frontend.protocols=THRIFT_BINARY,REST,FLIGHT_SQL
```

On start, `KyuubiServer` does roughly:

1. Read `FRONTEND_PROTOCOLS`
2. Map each name through `FrontendProtocols.withName`
3. Construct the matching `AbstractFrontendService`
4. Inject the **same** `BackendService` into every frontend

For Flight SQL that service is `KyuubiFlightSqlFrontendService`, which owns an Arrow `FlightServer`.

ACT 2 · WHY ARROW / FLIGHT SQL

The gap: engines are columnar, classic JDBC paths are row-shaped

Engines like Spark already execute and buffer results column-wise. Many SQL clients still pull cells through row-oriented APIs and rebuild columns in the tool.

> Arrow is the **shared memory / interchange layout** for those columns. Flight moves that layout over the network. Flight SQL puts a **SQL command surface** on top of Flight. Next slides unpack those three layers in order.

04 · APACHE ARROW

Arrow is a columnar in-memory / interchange format

ACT 2 — LAYER 1: THE DATA LAYOUT

Analytical engines already think in columns. JDBC/ODBC usually force:

1. engine materializes rows
2. client deserializes cell-by-cell
3. analytics tool rebuilds columns

Arrow avoids that reshape by defining:

- **Schema** — field names, logical types, nullability
- **RecordBatch** — fixed-length column vectors
- **Buffers** — validity bitmap, offsets, values

Flight SQL's value in Kyuubi is to expose those batches over a SQL API without teaching every client Spark/Trino internals.

04 · VISUAL

RecordBatch = schema + contiguous column vectors

Apache Arrow record batch layout

A Flight SQL stream is a sequence of these batches, not JDBC ResultSet rows.

Logical table

```
id:int32 | name:utf8 | amt:decimal
```

```
1 | alice | 10.50
2 | bob | 3.00
3 | carol | 7.25
```

Row-oriented clients walk left→right.

Arrow stores each column contiguously.

That is the unit Flight SQL streams.

RecordBatch = Schema + column vectors

Schema

```
Field(id, Int32), Field(name, Utf8), Field(amt, Decimal)
```

id vector

```
validity: 111
values:
  [1, 2, 3]
```

contiguous int32

name vector

```
validity: 111
offsets: 0,5,8,13
data: alicebobcarol
```

utf8 buffers

amt vector

```
validity: 111
values:
  [1050,300,725]
```

scale applied by type

Flight DoGet emits one or more RecordBatches with this schema.

05 · FLIGHT AND FLIGHT SQL

Two layers: transport vs SQL commands

ACT 2 – LAYERS 2 AND 3: MOVE BATCHES, THEN SPEAK SQL

Arrow Flight (transport):

- gRPC RPCs: `GetFlightInfo`, `DoGet`, `DoAction`, …
- Payload: Arrow IPC record batches
- Knows how to *move* Arrow data

Arrow Flight SQL (SQL protocol on Flight):

- Protobuf commands such as `CommandStatementQuery`
- Metadata commands: catalogs / schemas / tables / SQL info / XDBC types
- Producer interface: `FlightSqlProducer`

Kyuubi implements Flight SQL by adapting those commands onto existing BackendService session/operation APIs.

05 · VISUAL

Flight transport vs Flight SQL command surface

Arrow Flight vs Arrow Flight SQL

Flight is the transport. Flight SQL is the SQL command protocol on top of that transport.

Arrow Flight (transport)

```
DoGet(Ticket) → stream RecordBatch
```

```
GetFlightInfo(Descriptor) → FlightInfo
```

```
DoAction / ListFlights / Handshake
```

gRPC + Arrow IPC payload

Knows how to move Arrow data.

Does not define SQL semantics.

Arrow Flight SQL (commands)

```
CommandStatementQuery
```

```
CommandGetTables / GetCatalogs / ...
```

```
CommandGetSqlInfo / XdbcTypeInfo
```

```
CancelQuery / ClosePreparedStatement*
```

Defines SQL metadata and statement RPCs.

Implemented by FlightSqlProducer.

* prepared statements: not implemented yet in Kyuubi

06 · WHY INTEGRATE INTO KYUUBI

What Flight SQL buys when Kyuubi is the gateway

ACT 3 — PRODUCT MOTIVATION BEFORE IMPLEMENTATION DETAIL

Without Kyuubi, every Arrow client must learn each engine's API, auth, and lifecycle.

With Kyuubi + Flight SQL:

- **One SQL endpoint** for Arrow-native clients (`ADBC` , Java `FlightSqlClient` , …)
- **Reuse** Kyuubi auth, sessions, cancel/close, metrics, HA
- **Engine neutrality** — Spark today; Trino/Flink/Hive/JDBC through the same BackendService
- **Columnar results** — less row-oriented serialization than classic JDBC paths

07 · WHAT THIS TASK ADDED

New protocol: `FLIGHT_SQL`

ACT 3 – SCOPE OF THE IMPLEMENTATION

Implemented pieces:

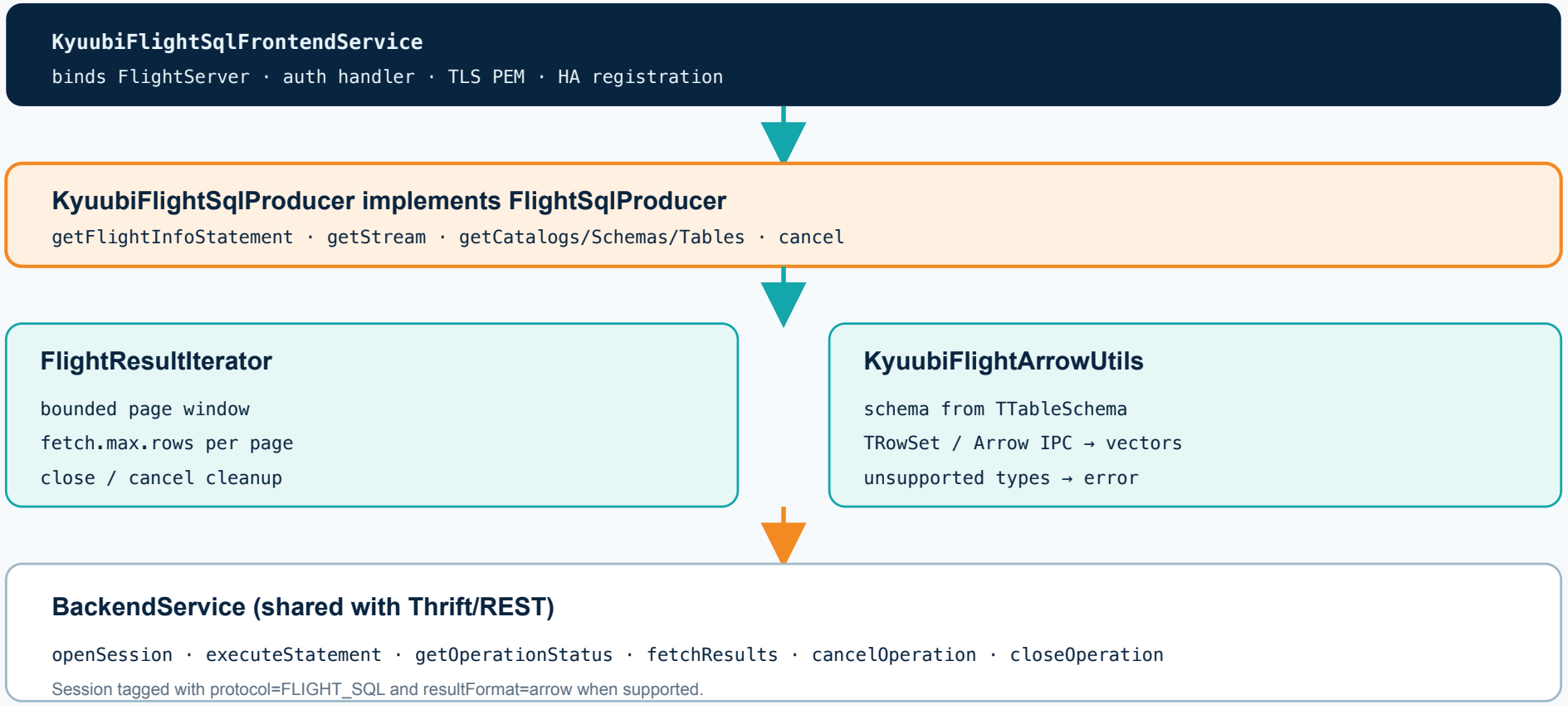
1. **Frontend lifecycle** – bind host/port, connection URL, auth, TLS, shutdown
2. **Producer bridge** – sessions, statements, metadata, tickets, paging, cancel
3. **Bounded streaming** – `FlightResultIterator` with `fetch.max.rows`
4. **Type conversion** – Spark Arrow IPC path + shared Thrift→Arrow converter
5. **Auth/TLS** – NONE, Basic/LDAP, SPNEGO→Bearer, PEM TLS
6. **HA** – `kyuubi_flight` namespace, node-affine tickets
7. **Metrics** – connection / operation / stream counters

07 · VISUAL

Class stack: frontend → producer → BackendService

Kyuubi Flight SQL class stack

The producer is a BackendService adapter — not an engine-specific connector.



08 · STATEMENT LIFECYCLE

One query = execute + doGet

ACT 3 – HOW EXECUTE AND DOGET MAP ONTO BACKENDSERVICE

Concrete call sequence:

1. Client calls Flight SQL `execute(sql)`
2. Producer opens a Kyuubi session (`protocol=FLIGHT_SQL`)
3. Producer calls `executeStatement` on BackendService
4. Engine runs SQL; operation handle becomes ready
5. Producer returns `FlightInfo` with an opaque **Ticket**
6. Client calls `doGet(ticket)`
7. Producer loops `fetchResults(FETCH_NEXT, maxRows)`
8. Each page becomes one or more Arrow **RecordBatches**

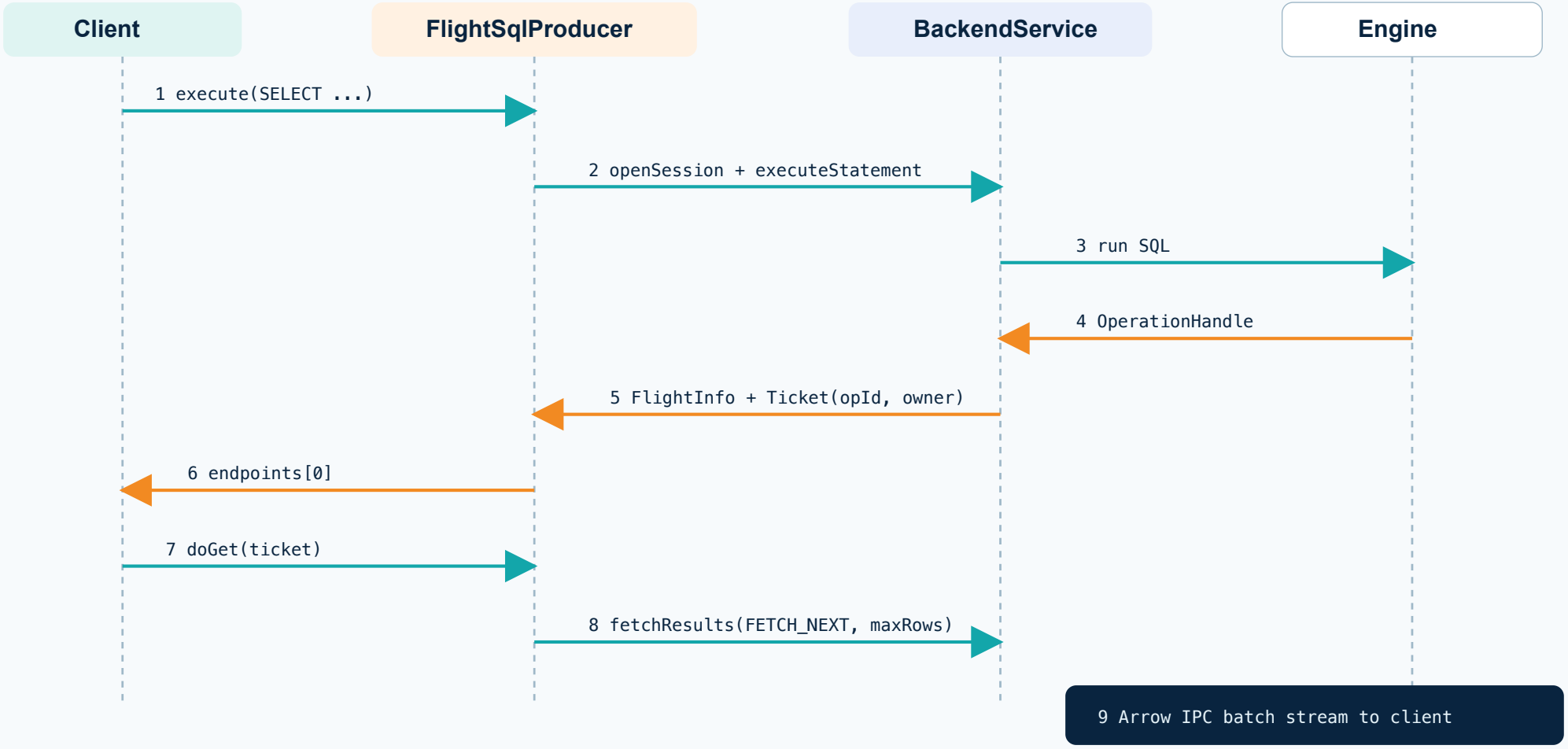
The ticket also carries **owner endpoint** information for HA.

08 · VISUAL

Statement lifecycle across Client / Producer / Backend / Engine

Flight SQL statement lifecycle in Kyuubi

Two-phase: create operation (execute) then stream pages (doGet).



09 · CONFIGURATION

Config keys that matter

ACT 3 – HOW OPERATORS TURN THE FRONTEND ON

Enablement:

```
kyuubi.frontend.protocols=THRIFT_BINARY,REST,FLIGHT_SQL
```

Flight-specific:

```
kyuubi.frontend.flight.sql.bind.host=0.0.0.0
kyuubi.frontend.flight.sql.bind.port=10299
kyuubi.frontend.flight.sql.fetch.max.rows=1000
kyuubi.frontend.flight.sql.ssl.enabled=false
kyuubi.frontend.flight.sql.ssl.cert.file=/path/cert.pem
kyuubi.frontend.flight.sql.ssl.key.file=/path/key.pem
kyuubi.frontend.flight.sql.token.ttl=PT2H
kyuubi.ha.flight.sql.namespace=kyuubi_flight
kyuubi.operation.result.format=arrow
```

09 · VISUAL

Available Flight SQL configuration keys

Available Flight SQL configuration keys

Enable via kyuubi.frontend.protocols; Flight knobs under kyuubi.frontend.flight.sql.*

Enablement

kyuubi.frontend.protocols = …,FLIGHT_SQL

Flight SQL

kyuubi.frontend.flight.sql.bind.host

kyuubi.frontend.flight.sql.bind.port (default 10299)

kyuubi.frontend.flight.sql.fetch.max.rows (default 1000)

kyuubi.frontend.flight.sql.ssl.enabled

kyuubi.frontend.flight.sql.ssl.cert.file / .ssl.key.file

kyuubi.frontend.flight.sql.token.ttl (default PT2H)

Related

kyuubi.ha.flight.sql.namespace (default kyuubi_flight)

kyuubi.operation.result.format=arrow

10 · STREAMING

Bounded pages, not whole-result materialization

ACT 3 – AFTER FLIGHTINFO, HOW ROWS LEAVE THE SERVER SAFELY

`FlightResultIterator` is the memory boundary:

- Requests at most `kyuubi.frontend.flight.sql.fetch.max.rows` rows per page (default **1000**)
- Retains **only the current page / batch**
- Releases previous page before fetching next
- Propagates cancel/close to BackendService
- Relies on Flight/gRPC backpressure while writing batches

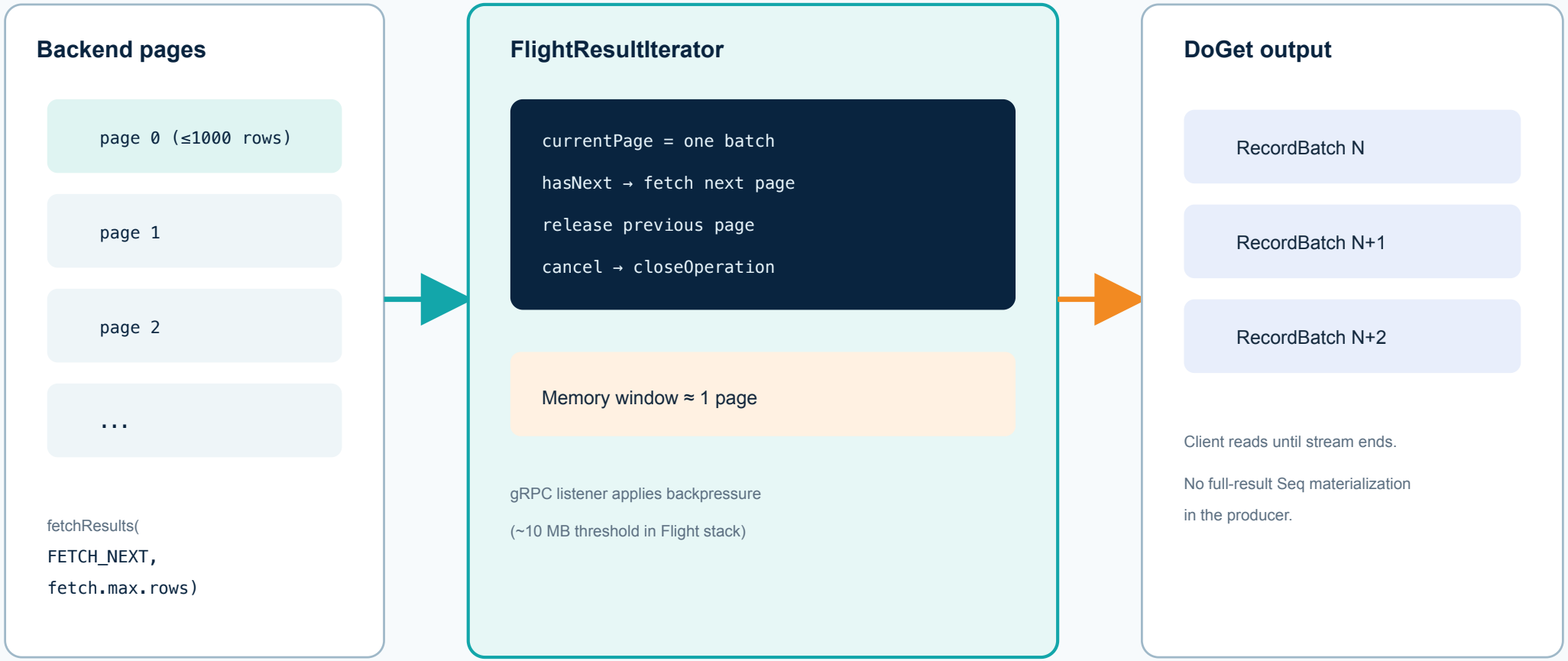
This is intentional: the producer must not collect the entire result as a Scala `Seq`.

10 · VISUAL

Page → iterator window → DoGet RecordBatches

Bounded result streaming

FlightResultIterator keeps only the current backend page / Arrow batch resident.



11 · TYPES

Cross-engine Arrow conversion contract

ACT 3 – WHAT THOSE RECORDBATCHES CONTAIN

Two paths into the same Flight schema:

1. **Spark** – prefers Arrow IPC inside the backend `TRowSet` when `operation.result.format=arrow`
2. **Trino / Flink / Hive / JDBC** – shared converter from columnar/ordinary Thrift pages into Arrow vectors

Supported primitives: bool, integer widths, float/double, decimal, date, timestamp, string, binary.

11 · VISUAL

Spark native IPC vs shared Thrift→Arrow bridge

Engine → Arrow conversion paths

One Flight schema; two conversion strategies depending on engine capability.

Spark

Native Arrow IPC

carried inside TRowSet

`operation.result.format=arrow`

Preferred path for Flight SQL demos.

Still crosses BackendService;
not engine-direct Flight.

KyuubiFlightArrowUtils

```
TTableSchema → Fields
TRowSet columns → vectors
populateRootFromRowSet
write IPC / VectorSchemaRoot
```

Supported primitives

bool, ints, float/double, decimal,
date, timestamp, string, binary

Same Arrow schema is exposed
to the Flight SQL client.

Trino / Flink / Hive / JDBC

Columnar / row TRowSet

no native Arrow IPC

Shared converter builds

Arrow vectors page-by-page.

Same Flight SQL client contract
regardless of engine.

12 · VISUAL

Auth modes and bearer token handoff

Authentication and TLS on Flight SQL

Flight authorization headers map onto Kyuubi authentication providers.

NONE / NOSASL

No Authorization header

optional x-user-name

session user = anonymous

Basic / LDAP

`Authorization: Basic ...`

validated by Kyuubi auth

then openSession(user)

Kerberos

`Authorization: Negotiate`

SPNEGO bootstrap

issues Bearer token

TLS

`ssl.enabled=true`

`cert.file + key.file`

URI: grpc+tls://

Token flow after Basic / SPNEGO

Local Docker demo uses NONE — no credentials, grpc:// (not grpc+tls://).

PEM files can also be materialized from shared kyuubi.frontend.ssl.keystore.* when cert/key paths are unset.

13 · HIGH AVAILABILITY

Separate discovery namespace; node-affine tickets

ACT 3 – DISCOVERY YES; MID-QUERY FAILOVER NO

Flight endpoints register under:

```
kyuubi.ha.flight.sql.namespace=kyuubi_flight
```

That namespace is **distinct** from Thrift and Spark Connect namespaces so clients cannot mix gRPC Flight endpoints with JDBC discovery.

Ticket contract:

- `FlightInfo` advertises the **owning** endpoint
- Ticket encodes operation id + owner identity
- `doGet` / cancel / close must hit that owner
- **No transparent mid-query failover**

Sticky routing (or follow advertised endpoint) is required.

13 · VISUAL

Namespace separation and owner-sticky tickets

HA: dedicated namespace + node-affine tickets

Flight discovery is separate from Thrift and Spark Connect. Active streams stick to the owning node.

ZooKeeper namespaces

```
kyuubi.ha.namespace (Thrift)
kyuubi.ha.spark.connect... (SC)
kyuubi.ha.flight.sql.namespace
```

Registered Flight nodes

node-A :10299

owner of ticket T1

advertised in FlightInfo

node-B :10299

not owner of T1

doGet(T1) → reject/stale

Client routing contract

1. execute() returns FlightInfo with endpoints = [owner host:port]
2. Ticket encodes operation id + owning node identity
3. doGet / cancel / close must target that owner
4. Transparent mid-query failover across nodes is not supported

Load balancers need sticky routing (or clients must follow advertised endpoint).

13B · HA REALITY CHECK

What Flight SQL HA can and cannot do

CAN

- Register Flight endpoints under `kyuubi_flight`
- Remove a dead node from ZooKeeper discovery
- Serve **new** sessions on the surviving Kyuubi
- Keep running Spark SQL (and `s3a://` reads) on the survivor
- Isolate Flight discovery from Thrift / Spark Connect namespaces

CANNOT

- Transparent **mid-query / mid-stream** failover
- Automatically **reuse the same** Kyuubi `SessionHandle` on another node
- Migrate sticky Flight tickets after `FlightInfo`
- Pretend a load balancer without sticky routing is safe

Compare: **Spark Connect** has client-side `FailoverManagedChannel` + `ReattachExecute` and a shared token store. Thrift/REST get discovery HA + shared engines, but still open a **new** session after reconnect.

13B · VISUAL

Frontend HA capability matrix

Frontend HA capability matrix

What fails over when a Kyuubi server dies — by frontend protocol.

Capability	FLIGHT_SQL	THRIFT / REST	SPARK_CONNECT
ZK discovery / remove dead node	YES (kyuubi_flight)	YES (kyuubi)	YES (kyuubi_sc)
New session on surviving node	YES	YES (reconnect)	YES
Same Kyuubi session reused	NO	NO	YES (token store)
Mid-query / stream failover	NO (sticky ticket)	NO	YES (ReattachExecute)

Demo implication

Stop kyuubi-1 → ZK drops :10299 → new Spark/MinIO SQL on :10300 works; in-flight Flight tickets on :10299 die.

14 · OPS AND METRICS

Capability boundary and observability

ACT 3 – WHAT TO PROMISE CLIENTS AND WHAT TO SCRAPE

Supported

- SQL statements, bounded Arrow streaming, cancel/cleanup
- catalogs / schemas / tables / table types / SQL info / XDBC types
- NONE / Basic / SPNEGO→Bearer / PEM TLS

Not yet (`UNIMPLEMENTED`)

- prepared statements, ingestion, Substrait, transactions/savepoints
- transparent HA mid-stream failover

Metrics (Prometheus `:10019`):

- `kyuubi.flight.sql.connection.{opened,total,failed}`
- `kyuubi.flight.sql.operation.{opened,total,failed,cancelled}`
- `kyuubi.flight.sql.stream.{batches,rows,bytes}`